

ASSIGNMENT – 12

NAME : M.AKSHAY PATEL

HALL TICKET NO : 2303A51745

BATCH : 11

Question :

Hyperledger Fabric Problem Solving Objective To design and implement a simple Hyperledger Fabric smart contract (chaincode) that manages a personal asset portfolio on a blockchain network. The practical focuses on problem-solving using Fabric concepts such as assets, transactions, ledger state, and queries. Problem Statement Implement a basic blockchain-based portfolio system where a user can:

- Create assets (e.g., certificates, projects, or tokens)
- View all assets stored on the blockchain
- Transfer asset ownership between users

This practical demonstrates how Hyperledger Fabric solves realworld problems like data immutability, transparency, and secure ownership transfer.

Requirements Software Requirements

- Operating System: Windows / Linux / macOS
- Node.js (v16 or later)
- Docker & Docker Compose
- Git
- VS Code (with Docker and JavaScript extensions)

Blockchain Platform

- Hyperledger Fabric test network

CODE :

```
import tkinter as tk from tkinter import messagebox

import random import time class DeFi:    def
__init__(self):        self.balances = {}    def gas(self):
return round(random.uniform(0.0001, 0.001), 6)

def delay(self):        time.sleep(0.3)    def
deposit(self, user, amount):        if amount <= 0:
        return "Invalid amount"

self.delay()        gas = self.gas()    if
user not in self.balances:
self.balances[user] = 0
self.balances[user] += amount
return f"Deposit Success\nGas
Fee: {gas}"    def withdraw(self,
user, amount):        if user not in
self.balances:        return "User
not found"        if
self.balances[user] < amount:
return "Insufficient balance"

self.delay()        gas = self.gas()
```

```

self.balances[user] -= amount

return f"Withdraw Success\nGas
Fee: {gas}"

def balance(self,
user):

    return self.balances.get(user,
0)

class App:
    def __init__(self,
root):

        self.defi = DeFi()
        self.root = root
        self.root.title("DeFi Menu Interface")

        self.root.geometry("500x300")
        menubar = tk.Menu(root)
        root.config(menu=menubar)
        action_menu = tk.Menu(menubar, tearoff=0)
        menubar.add_cascade(label="Actions", menu=action_menu)
        action_menu.add_command(label="Deposit", command=self.deposit_window)
        action_menu.add_command(label="Withdraw", command=self.withdraw_window)
        action_menu.add_command(label="Check Balance", command=self.balance_window)

        self.label = tk.Label(root, text="Select an action from menu",
                                font=("Arial", 14))

        self.label.pack(pady=50)

    def
deposit_window(self):
        self.popup("Deposit")

    def
withdraw_window(self):
        self.popup("Withdraw")

    def
balance_window(self):
        self.popup("Balance")

    def
popup(self, action):
        win =
tk.Toplevel(self.root)
        win.title(action)
        tk.Label(win,
text="User").pack()
        user_entry = tk.Entry(win)
        user_entry.pack()

        if action !=
"Balance":

```

```

        tk.Label(win, text="Amount").pack()
amount_entry = tk.Entry(win)
amount_entry.pack()    def submit():
    user = user_entry.get()
if action == "Deposit":
    try:
        amount = float(amount_entry.get())
result = self.defi.deposit(user, amount)
except:
    result = "Invalid input"
elif action == "Withdraw":
    try:
        amount = float(amount_entry.get())
result = self.defi.withdraw(user, amount)
    except:
        result = "Invalid input"
else:
    result = f"Balance: {self.defi.balance(user)} ETH"
messagebox.showinfo("Result", result)    tk.Button(win,
text="Submit", command=submit).pack(pady=10) if __name__ ==
 "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()

```

IMPLEMENTATION :

```

C:\Users> Rushi > Downloads > guipy > Defi
1 import tkinter as tk
2 from tkinter import messagebox
3 import random
4 import time
5 class Defi:
6     def __init__(self):
7         self.balances = {}
8
9     def gas(self):
10        return round(random.uniform(0.0001, 0.001), 6)
11
12    def delay(self):
13        time.sleep(0.3)
14
15    def deposit(self, user, amount):
16        if amount <= 0:
17            return "Invalid amount"
18
19        self.delay()
20        gas = self.gas()
21
22        if user not in self.balances:
23            self.balances[user] = 0
24
25        self.balances[user] += amount
26        return f"Deposit Success\nGas Fee: {gas}"
27
28    def withdraw(self, user, amount):
29        if user not in self.balances:
30            return "User not found"
31
32        if self.balances[user] < amount:
33            return "Insufficient balance"

```

```

C:\Users> Rushi > Downloads > guipy > Defi
43 class App:
44     def popup(self, action):
45         user_entry = tk.Entry(win)
46         user_entry.pack()
47
48         if action != "Balance":
49             tk.Label(win, text="Amount").pack()
50             amount_entry = tk.Entry(win)
51             amount_entry.pack()
52
53     def submit():
54         user = user_entry.get()
55
56         if action == "Deposit":
57             try:
58                 amount = float(amount_entry.get())
59                 result = self.defi.deposit(user, amount)
60             except:
61                 result = "Invalid input"
62         elif action == "Withdraw":
63             try:
64                 amount = float(amount_entry.get())
65                 result = self.defi.withdraw(user, amount)
66             except:
67                 result = "Invalid input"
68         else:
69             result = f"Balance: {self.defi.balance(user)} ETH"
70             messagebox.showinfo("Result", result)
71             tk.Button(win, text="Submit", command=submit).pack(pady=10)
72
73 if __name__ == "__main__":
74     root = tk.Tk()
75     app = App(root)
76     root.mainloop()

```

OUTPUT :

